

The Hitchhiker's Compiler: un compilador de HULK en Rust con genericidad, interfaces y recolección de basura

Kevin Alejandro Torres Perera, Lianny de la Caridad Revée Valdivieso, and
Jocdan Lismar López Mantecón

Grupos 311 y 312, Ciencias de la Computación,
Facultad de Matemática y Computación,
Universidad de La Habana

<https://github.com/MatCom-Developing-Team-10/The-Hitchhikers-Compiler>



Resumen Este trabajo presenta *The Hitchhiker's Compiler*, un compilador completo para el lenguaje HULK (*Havana University Language for Kompilers*) implementado en Rust como proyecto integrador de las asignaturas Compilación y Lenguajes de Programación. El compilador cubre el pipeline clásico —análisis léxico, sintáctico LALR(1), análisis semántico con verificación de tipos, descenso a una representación intermedia y ejecución sobre una máquina virtual de pila— organizado como un *workspace* de siete *crates* con dependencias unidireccionales que materializan los principios SOLID y DRY. El análisis semántico se estructura en una secuencia de pasadas —cuatro fundamentales más inferencia de tipos (A.9) y verificación de interfaces— para satisfacer la visibilidad global de declaraciones que exige HULK, acumula diagnósticos en lugar de abortar y emplea un patrón *poison* (`Type::Error`) para suprimir cascadas de errores. Sobre esta base, el equipo diseñó y verificó tres extensiones originales al lenguaje, fundamentadas en un estudio comparativo con lenguajes de programación históricos: (i) **genericidad** con parámetros de tipo [T] resueltos por *type erasure* e invarianza, en la línea de Java y Scala; (ii) **interfaces** con cláusulas `implements/extends` que admiten conformidad tanto nominal como estructural, combinando los modelos de Java y Go; y (iii) un **recolector de basura** *mark-and-sweep* con *heap* indexado por *handles* que reclama ciclos, inspirado en la tradición iniciada por Lisp. Se describen la arquitectura, las decisiones de diseño, las reglas semánticas de cada extensión y su implementación real, acompañadas de una evaluación mediante una batería de pruebas unitarias, de integración y *end-to-end*, junto con un análisis crítico de ventajas, limitaciones y mejoras futuras. El resultado es un compilador correcto, evaluable de forma automática y extensible sin modificar la lógica existente.

Keywords: compiladores, HULK, Rust, sistemas de tipos, polimorfismo paramétrico, interfaces, recolección de basura, máquina virtual

1. Introducción

La construcción de un compilador es, posiblemente, el ejercicio integrador por excelencia de las Ciencias de la Computación: reúne teoría de lenguajes formales, estructuras de datos, sistemas de tipos, diseño de software y arquitectura de máquinas en un único artefacto verificable. Un *compilador* es un programa que traduce texto escrito en un lenguaje fuente a un lenguaje objeto, preservando su significado [1]. Clásicamente, la traducción se descompone en un *frontend* —análisis léxico, sintáctico y semántico, que comprende el programa y rechaza los incorrectos— y un *backend* —síntesis, que genera código o lo ejecuta— unidos por una o varias representaciones intermedias.

Este informe documenta *The Hitchhiker's Compiler*, un compilador del lenguaje HULK desarrollado en Rust. El proyecto persigue un doble objetivo académico. Desde la asignatura **Compilación**, demuestra el dominio del pipeline completo, desde el flujo de caracteres hasta la ejecución, con un reporte de errores posicionado y la conformidad con el contrato de evaluación automática de la cátedra (*matcom/compilers*). Desde la asignatura **Lenguajes de Programación**, justifica y materializa una extensión original del lenguaje, cuyas decisiones de diseño se sustentan en la comparación científica con lenguajes históricos estudiados en el curso.

1.1. El lenguaje HULK

HULK (*Havana University Language for Kompilers*) es un lenguaje didáctico, fuertemente tipado, orientado a objetos con herencia simple, y de naturaleza *expressional*: prácticamente toda construcción —incluidos `if` y bloques— produce un valor [8]. Un programa HULK es una secuencia de declaraciones de tipos y funciones seguida de una expresión principal. El lenguaje ofrece literales numéricos, booleanos y de cadena; operadores aritméticos, booleanos con cortocircuito, de comparación y de concatenación (`@`, `@@`); enlaces locales `let ... in`; condicionales `if/elif/else`; ciclos `while` y `for`; funciones de primer orden y recursión; declaración de tipos con constructores parametrizados, atributos, métodos con despacho virtual, herencia (`inherits`) y despacho al padre (`base`); y los operadores de tipo `is` y `as`. La especificación mínima exigida abarca las secciones A.1–A.8 del libro del curso, hasta la verificación de tipos.

1.2. Tesis y contribuciones

La tesis de este trabajo es que un compilador educativo, diseñado con una arquitectura modular y un sistema de tipos sólido, constituye una base natural y económica para incorporar características avanzadas de lenguajes de programación sin reescribir la lógica existente. Para demostrarlo, el equipo extendió HULK con tres características, cada una elegida por su relevancia teórica y su lugar en la historia de los lenguajes:

1. **Genericidad** (polimorfismo paramétrico): parámetros de tipo con sintaxis [T] resueltos por *type erasure*, con conformidad invariante. Se contrastan con el *erasure* de Java y la monomorfización de Rust.
2. **Interfaces**: contratos de métodos con herencia múltiple entre interfaces (*extends*) y conformidad que admite tanto la vía nominal (*implements*) como la estructural (*protocol-style*). Se sitúan entre el modelo nominal de Java y el estructural de Go.
3. **Recolector de basura** *mark-and-sweep* con *heap* indexado por *handles*, capaz de reclamar ciclos, en la tradición del *tracing* iniciada por McCarthy en Lisp.

El resto del documento se organiza como sigue. La Sección 2 desarrolla el marco teórico que sustenta el proyecto. La Sección 3 describe la arquitectura y la implementación de cada fase del compilador. La Sección 4 —núcleo del aporte original— define, analiza y demuestra las tres extensiones. La Sección 5 resume la estrategia de pruebas y calidad, y la Sección 7 ofrece conclusiones, comparativas, críticas y trabajo futuro.

2. Marco teórico

2.1. Anatomía de un compilador

Siguiendo a Aho et al. [1], la traducción se modela como una secuencia de fases que transforman progresivamente la representación del programa. El *análisis léxico* agrupa los caracteres en *tokens*; el *análisis sintáctico* reconoce la estructura gramatical y construye un árbol de sintaxis abstracta (AST); el *análisis semántico* verifica reglas que la gramática no captura —sobre todo el sistema de tipos y la resolución de nombres—; y la *síntesis* genera código intermedio y final. La Figura 3 muestra la instancia concreta de este esquema en nuestro proyecto.

2.2. Análisis léxico y autómatas finitos

El analizador léxico se especifica naturalmente con expresiones regulares, cuyo reconocimiento equivale a un autómata finito determinista (AFD). Generadores modernos como *logos* [19] traducen anotaciones declarativas sobre un tipo enumerado a un AFD eficiente en tiempo de compilación, evitando el costo de construir el autómata en tiempo de ejecución. La salida es un flujo de *lexemas* clasificados, cada uno acompañado de su posición en la fuente, base indispensable para los diagnósticos.

2.3. Análisis sintáctico LR(1) y LALR(1)

La sintaxis de los lenguajes de programación se describe con gramáticas libres de contexto. Entre los métodos de análisis ascendente, las gramáticas LR(1) [13] reconocen una clase amplia de lenguajes deterministas con un símbolo de anticipación. Las gramáticas **LALR(1)** [7] fusionan estados LR(1) con el mismo

núcleo, reduciendo drásticamente el tamaño de la tabla de análisis a cambio de una pérdida mínima de poder expresivo; este es el equilibrio que adoptan generadores como yacc, bison y, en el ecosistema Rust, *LALRpop* [15]. La *precedencia* y la *asociatividad* de los operadores se resuelven estratificando la gramática en niveles, técnica que evita ambigüedades sin recurrir a directivas externas.

2.4. El árbol de sintaxis abstracta

El AST es la columna vertebral del compilador: una representación en memoria, independiente de la sintaxis concreta, que las fases posteriores recorren y anotan. Un buen diseño de AST equilibra *fidelidad* (preservar la información necesaria, incluidos los *spans* de origen para diagnosticar) y *economía* (no arrastrar detalles redundantes). En lenguajes algebraicos como Rust, el AST se modela idiomáticamente con tipos suma (`enum`), lo que hace del recorrido un *pattern matching* exhaustivo y verificado por el compilador.

2.5. Sistemas de tipos, conformidad y subtipado

Un *sistema de tipos* es un método sintáctico tratable para demostrar la ausencia de ciertos comportamientos erróneos clasificando las frases del lenguaje según los valores que computan [18]. En un lenguaje orientado a objetos con herencia, la relación central es la de *subtipado* o *conformidad*: $S \leq T$ significa que un valor de tipo S puede usarse en todo contexto que espere un T (principio de sustitución de Liskov [14]). HULK organiza sus tipos en una jerarquía de herencia simple con un tipo tope `Object`; la conformidad se calcula recorriendo la cadena de ancestros. Cuando una expresión compuesta como `if` tiene ramas de tipos distintos, su tipo es el *ancestro común más bajo* (LCA) de las ramas, análogo a la noción de *join* en un retículo de tipos.

Formalmente, la relación de conformidad $\leq$ que implementa nuestro verificador (función `conforms`) se rige por las reglas de la Figura 2. La regla (TOP) hace de `Object` el supremo; (REFL) y (TRANS-INH) dan reflexividad y el ascenso por la cadena de herencia; (POISON) convierte a `Type::Error` en comodín bidireccional para suprimir cascadas; y (VEC-ITER) expresa que un vector conforma al iterable tipado covariantemente en el elemento. Las reglas de genericidad e interfaces (INV y IFACE) se desarrollan en la Sección 4.

El cálculo del LCA de dos tipos A y B se reduce a recolectar la cadena de ancestros de A (incluido él mismo) hasta `Object` y devolver el primer ancestro de B que pertenezca a ese conjunto; como la herencia es simple y acíclica, el procedimiento siempre termina y, en el peor caso, en `Object` (Algoritmo 1). Esta operación es la que fija el tipo de un `if` con ramas heterogéneas y el tipo de elemento de un literal de vector.

2.6. Polimorfismo

Cardelli y Wegner [5] distinguen el polimorfismo *universal* —paramétrico (genericidad) y de inclusión (subtipado)— del *ad-hoc* —sobrecarga y coerción—. El

```

Algoritmo 1: LCA( $A, B$ )
 $\text{anc} \leftarrow \{A\} \cup \{\text{ancestros de } A \text{ hasta } \text{Object}\}$ 
para cada  $C$  en la cadena  $B, \text{padre}(B), \dots, \text{Object}$ :
    si  $C \in \text{anc}$  entonces devolver  $C$ 
devolver  $\text{Object}$       (siempre alcanzable: es el supremo)

```

Figura 1. Cálculo del ancestro común más bajo (*join* en el retículo de herencia).

$(\text{TOP}) \quad T \leq \text{Object} \quad (\text{REFL}) \quad T \leq T \quad (\text{POISON}) \quad \text{Error} \leq T \wedge T \leq \text{Error}$
$(\text{TRANS-INH}) \quad \frac{U \text{ inherits } V \quad V \leq T}{U \leq T} \quad (\text{VEC-ITER}) \quad \frac{S \leq T}{S[] \leq T*}$

Figura 2. Reglas de conformidad $\leq$ del verificador de tipos (núcleo A.8.4). Las reglas para genericidad (invarianza) e interfaces (nominal/estructural) aparecen en la Sección 4.

polimorfismo paramétrico permite escribir código uniforme sobre una familia de tipos mediante variables de tipo. Existen dos estrategias de realización:

- **Borrado de tipos** (*type erasure*): los parámetros de tipo existen solo en compilación y se eliminan antes de ejecutar. Es la vía de Java [4] y mantiene una única representación en tiempo de ejecución, a costa de perder información de tipo en *runtime*.
- **Monomorfización**: el compilador genera una copia especializada del código para cada instanciación concreta. Es la vía de C++ y Rust [12]: más rápida y con información de tipo completa, pero a costa de mayor tamaño de código.

2.7. Subtipado nominal frente a estructural

Un sistema **nominal** establece la relación de subtipo solo cuando se declara explícitamente (**implements**, **extends**): el nombre manda. Un sistema **estructural** la deduce de la forma: un tipo conforma a una interfaz si posee los miembros requeridos, haya o no declaración. Java, Kotlin y Swift son nominales; Go y TypeScript son estructurales. El primero evita acoplamientos accidentales y produce mensajes más claros; el segundo favorece el desacoplamiento y la composición. Como se verá, nuestra extensión de interfaces ofrece *ambas* disciplinas de forma controlada.

2.8. Representación intermedia y máquinas de pila

Entre el frontend y la ejecución, los compiladores introducen una representación intermedia (IR) que desacopla el lenguaje fuente de la máquina objetivo [2]. Una *máquina de pila* es un modelo de ejecución en el que los operandos se empujan y extraen de una pila; su IR es un *bytecode* compacto y fácil de generar a partir

de un AST mediante un recorrido post-orden, razón por la cual la adoptan intérpretes como los de Python, la JVM o Lua [17]. Es el modelo que elegimos para el *backend*.

2.9. Gestión automática de memoria

La *recolección de basura* (GC) automatiza la liberación de memoria inalcanzable [10]. Existen dos grandes familias:

Conteo de referencias. Cada objeto mantiene un contador de referencias entrantes; al llegar a cero se libera. Es incremental y simple, pero **no reclama ciclos** por sí solo, lo que obliga a añadir un detector de ciclos como el de Bacon y Rajan [3] (la estrategia de CPython).

Rastreo (*tracing*). Se parte de un conjunto de *raíces* (pila, variables globales) y se visitan transitivamente los objetos alcanzables; lo no alcanzado es basura. El algoritmo **mark-and-sweep**, introducido por McCarthy para Lisp [16,6], recorre y marca lo vivo (*mark*) y luego barre lo muerto (*sweep*). Reclama ciclos de forma natural, a costa de pausas *stop-the-world*.

Refinamientos posteriores —copia semiespacial, recolección generacional [11] y colectores concurrentes como los de Go y la JVM— mejoran la latencia y el rendimiento, pero exceden el alcance de un compilador educativo. Lenguajes como Rust prescinden de GC mediante un sistema de *ownership* verificado estáticamente [12]; HULK, al permitir mutación compartida a través de objetos, sí requiere gestión automática.

3. Arquitectura y desarrollo

3.1. Organización del *workspace*

El compilador es un *workspace* de Cargo con siete *crates*, conectados por el flujo de datos `lexer -> ast <- parser -> semantic -> ir -> vm` y orquestados por el binario `hulkc` (Figura 3). La dirección de las dependencias es estrictamente unidireccional: ninguna fase posterior puede contaminar a una anterior, lo que encarna el principio de inversión de dependencias y permite razonar sobre cada fase de forma aislada.

El *crate* `hulk-ast` ocupa una posición central: tanto el parser que lo produce como el analizador semántico y el generador de IR que lo consumen dependen de él. Mantener el AST como única fuente de verdad evita la duplicación de definiciones y materializa el principio DRY: un nodo nuevo se define una sola vez y todas las fases lo ven de inmediato.

3.2. Análisis léxico (`hulk-lexer`)

El lexer se construye sobre *logos*, que genera un AFD eficiente a partir de anotaciones declarativas sobre un `enum Token`. Reconoce medio centenar de entradas

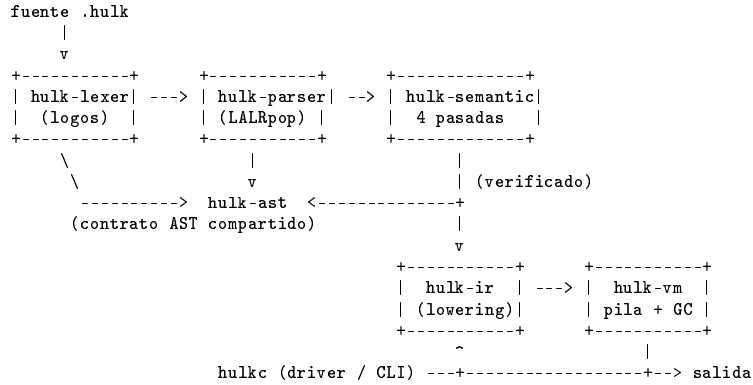


Figura 3. Pipeline de *The Hitchhiker's Compiler*. El AST (**hulk-ast**) es el contrato compartido entre el frontend y las fases posteriores; las dependencias fluyen en una única dirección.

Cuadro 1. Responsabilidad de cada *crate* del *workspace*.

Crate	Responsabilidad
hulk-ast	Nodos del AST, Span , operadores y declaraciones (contrato compartido).
hulk-lexer	Tokenización con <i>logos</i> ; adapta el flujo al protocolo de LALRpop.
hulk-parser	Gramática LALRpop $\rightarrow$ Program (AST).
hulk-semantic	Análisis semántico: nombres, <i>scopes</i> , herencia, tipos.
hulk-ir	Descenso del AST a <i>bytecode</i> de máquina de pila.
hulk-vm	Intérprete de <i>bytecode</i> con GC <i>mark-and-sweep</i> .
hulkc	<i>Driver</i> de línea de comandos; orquesta el pipeline.

léxicas (cerca de cincuenta reglas `##[token]/##[regex]`, incluidos los patrones de descarte de espacios y comentarios): palabras clave (`let`, `in`, `if`, `elif`, `else`, `while`, `for`, `function`, `type`, `inherits`, `new`, `is`, `as`, `true`, `false`, y las de las extensiones `interface`, `implements`, `extends`), operadores —incluidos los de concatenación `@` y `@@` y el de potencia—, delimitadores (entre ellos `[` y `]` añadidos para genericidad y vectores), literales e identificadores. Una decisión relevante es tratar `self` y `base` como identificadores ordinarios, dejando su semántica especial a fases posteriores. El lexer descarta espacios y comentarios de línea (`//`); no soporta comentarios de bloque. Expone un `Lexer<'input>` que adapta el iterador de *logos* al protocolo *triple* (`inicio`, `Token`, `fin`) que LALRpop exige, y representa los errores con `LexError`, que guarda los *offsets* de byte del fragmento ofensivo.

3.3. Análisis sintáctico (hulk-parser)

El parser usa LALRpop. La gramática completa vive en `grammar.lalrpop` y se traduce a Rust mediante un *build script*. La API pública es una única función

`parse(source) -> Result<Program, ParseError>` que invoca el lexer internamente, de modo que el consumidor solo interactúa con texto y un AST. La gramática define **doce niveles de precedencia**, desde la asignación destructiva `:=` en el nivel más bajo hasta el acceso a miembros y las llamadas en el más alto; el operador de potencia `^` es asociativo por la derecha y los unarios tienen mayor precedencia que la multiplicación pero menor que la potencia, conforme a la especificación. Dos decisiones merecen mención. El `let` con múltiples enlaces se *desazucara* a `Let` unarios anidados, eliminando toda lógica de enlace paralelo del verificador. Y el `if` lleva siempre una rama `else` obligatoria: como en HULK el `if` es una expresión que debe producir un valor en toda rama, una rama faltante haría su tipo indefinible. Cada nodo carga un `Span` (par de *offsets* lo/hi) capturado con los marcadores `@L/@R` de LALRpop. El Listado 1.1 muestra el desazucarado del `let` múltiple: una construcción superficial con varios enlaces se traduce a una cadena de `let` unarios cuyo alcance anidado reproduce exactamente la semántica secuencial de A.4.1, de modo que un enlace posterior puede referirse a uno anterior.

Listing 1.1. Desazucarado del `let` con múltiples enlaces a `let` unarios anidados.

```
// superficie (lo que escribe el programador)
let a = 1, b = a + 1, c = b * 2 in c
// AST tras el desazucarado (semantica equivalente)
let a = 1 in (let b = a + 1 in (let c = b * 2 in c))
```

3.4. El AST (hulk-ast)

El AST define un `Program` raíz que agrupa declaraciones de interfaces, tipos y funciones, más una expresión principal. Las expresiones se modelan con un `enum ExprKind` de más de veinte variantes (literales, operaciones binarias y unarias, variables, bloques, `let`, `if/elif/else`, `while`, `for`, llamadas a funciones y métodos, acceso y asignación de atributos, `new`, `is/as`, y los vectores `Vector`, `VectorComp` e `Index`). Cada `Expr` combina su `ExprKind` con un `Span`; los *offsets* de byte se convierten a línea/columna solo en el punto de reporte, evitando arrastrar información redundante por todo el árbol.

3.5. Análisis semántico (hulk-semantic)

El análisis semántico es el corazón del compilador y se organiza como una **secuencia de ocho pasadas** sobre el AST (Tabla 2), articulada en torno a cuatro pasadas *fundamentales* (`collect`, `sign`, `check_overrides`, `check_bodies`) a las que se intercalan tres pasadas de **inferencia de tipos** (A.9) y una de verificación de **interfaces** (extensión). El diseño multi-pasada responde a un requisito de HULK: todas las funciones y tipos son visibles con independencia del orden de declaración, por lo que es imposible verificar cuerpos en una sola pasada.

Las tres pasadas de inferencia —`infer_ctor_params`, `infer_params` e `infer_returns`, ejecutadas tras `sign`— van *más allá del mínimo exigido* (A.1–A.8): permiten

Cuadro 2. Las ocho pasadas del análisis semántico tal como las orquesta `analyze()`: cuatro fundamentales, tres de inferencia (A.9) y una de interfaces (extensión).

# Pasada	Propósito
1 <code>collect</code>	Registrar nombres de tipos; rechazar duplicados, herencia de <i>builtins</i> y ciclos.
2 <code>sign</code>	Llenar firmas de métodos y funciones; resolver tipos de parámetros anotados.
3 <code>infer_ctor_params</code>	(A.9.3) Inferir tipos de parámetros de constructor sin anotar.
4 <code>infer_params</code>	(A.9) Inferir tipos de parámetros de función/método sin anotar.
5 <code>infer_returns</code>	Inferir tipos de retorno sin anotar (por punto fijo).
6 <code>check_interfaces</code>	(extensión) Verificar que cada <code>implements</code> cubre los métodos requeridos.
7 <code>check_overrides</code>	Verificar que los <i>overrides</i> mantienen firma idéntica al padre.
8 <code>check_bodies</code>	Recorrer expresiones; inferir/verificar tipos; acumular errores.

omitir anotaciones de tipo en parámetros de constructor (A.9.3), en parámetros de función/método y en tipos de retorno, deduciéndolos por punto fijo a partir de los usos. Operan iterando hasta estabilizar las firmas (cada vuelta resuelve al menos un nivel de dependencia, de modo que un grafo acíclico converge en un número acotado de pasos). Esta capacidad se ejercita en las pruebas `a9_param_inference`, `a7_unannotated_point` y `ctor_inference`.

Una decisión central es el **manejo de errores por acumulación**: las pasadas nunca hacen *panic*; recogen todos los diagnósticos en un `Vec<SemError>` y los retornan juntos, lo que permite reportar múltiples problemas en una sola ejecución. Cada variante de `SemError` (más de quince: variable indefinida, tipo no coincidente, aridad incorrecta, nombre reservado, firma de *override* inconsistente, `NotAnInterface`, `NotIndexable`, etc.) carga un `Span` accesible por `span()`.

Listing 1.2. Algunas variantes de `SemError`; cada una porta su `Span` para diagnosticar con precisión.

```
#[derive(Debug, Error)]
pub enum SemError {
    #[error("undefined variable '{name}'")]
    UndefinedVariable { name: String, span: Span },
    #[error("type mismatch: expected '{expected}', found '{found}'")]
    Mismatch { expected: String, found: String, span: Span },
    #[error("wrong number of arguments: expected {expected}, found {found}")]
    Arity { expected: usize, found: usize, span: Span },
    #[error("method '{name}' overrides parent with a different signature")]
}
```

```
OverrideSignatureMismatch { name: String, span: Span },
// ... (cyclic inheritance, duplicate type, no such method, etc.)
}
```

Para evitar **cascadas de errores**, el verificador usa el patrón *poison* `Type::Error`. Cuando una subexpresión falla, recibe el tipo `Type::Error`, que la relación de conformidad trata como comodín: conforma con cualquier tipo y cualquier tipo conforma con él. Así, un único error profundo no genera docenas de errores derivados. El `enum Type` (Listado 1.3) modela los tipos primitivos, `Object`, los tipos de usuario, y —por las extensiones— `Generic`, `Param`, `Iterable` y `Vector`.

Listing 1.3. El `enum Type` con las variantes de las extensiones.

```
pub enum Type {
  Number, String, Boolean, Object,
  User(String),           // tipo de usuario no generico
  Generic(String, Vec<Type>), // List[Number], Map[String, Point]
  Param(String),          // parametro de tipo 'T' sin resolver
  Iterable(Box<Type>),     // T* (A.11.2)
  Vector(Box<Type>),       // T[] (A.12)
  Error,                  // poison: suprime cascadas
}
```

3.6. Representación intermedia y máquina virtual

Tras la verificación, `lower_program` desciende el AST a un *bytecode* para una máquina de pila. El *crate hulk-vm* es un intérprete que ejecuta las instrucciones sobre una pila de operandos y soporta el lenguaje completo: aritmética (con potencia y módulo), booleanos con cortocircuito, comparaciones, concatenación, control de flujo (`if/elif/else`, `while`, `for` sobre el protocolo `iterable next()/current()`), recursión con detección de desbordamiento de pila, orientación a objetos (herencia, constructores parametrizados, despacho virtual, `self`, `base`), y funciones *builtin* (`sqrt`, `sin`, `cos`, `exp`, `log`, `rand`, `print`, `range`). Más allá del mínimo, la VM implementa los **vectores** de A.12. Para soportar recursión profunda, el programa se ejecuta en un hilo dedicado con una pila de gran tamaño.

El conjunto de instrucciones (`enum Instr`) es deliberadamente pequeño y ortogonal (Tabla 3). Las constantes y las operaciones aritméticas, booleanas y de comparación trabajan exclusivamente sobre la pila; el control de flujo se expresa con etiquetas simbólicas (`Label`) y saltos (`Jump/JumpIfFalse`); los *scopes* léxicos se delimitan con `BeginScope/EndScope` y los enlaces con `BindVar`; y la orientación a objetos se reduce a `NewObject`, `GetField/SetField`, `CallMethod` (despacho dinámico por el tipo concreto) y `CallBase` (despacho estático al método del padre). Esta uniformidad hace que el descenso sea un recorrido post-orden directo del AST.

A modo de ilustración, la expresión principal del Listado 1.5 —`print((new Dog("Toby")).speak())`— desciende al *bytecode* del Listado 1.4. Nótese cómo

Cuadro 3. Conjunto de instrucciones del IR (`enum Instr`), agrupado por función.

Grupo	Instrucciones
Constantes	PushNum, PushBool, PushStr, PushNil
Pila	Pop, Dup
Aritmética	Add, Sub, Mul, Div, Pow, Mod, Neg
Lógica/comp.	And, Or, Not, Eq, Ne, Lt, Le, Gt, Ge
Cadenas	Concat, ConcatWs
Variables	LoadVar, StoreVar, BeginScope, BindVar, EndScope
Control	Label, Jump, JumpIfFalse, Call, Ret
Builtins	Print, Sqrt, Sin, Cos, Exp, Log, Rand
Objetos	NewObject, GetField, SetField, CallMethod, CallBase
Tipos	IsType, AsType
Vectores	MakeVector, Index, VecPush

`CallMethod` difiere la resolución al *runtime* (el objeto en la cima de la pila determina la implementación de `speak`), mientras que dentro de `Dog::speak` la llamada `base()` se compila como un `CallBase` que nombra estáticamente al tipo padre.

Listing 1.4. Descenso a *bytecode* de la expresión principal del Listado 1.5.

```
NewObject "Dog"      ; aloca Dog en el heap -> ObjectId en la pila
PushStr   "Toby"      ; argumento del constructor
CallMethod "<ctor>", 1 ; inicializa los campos
CallMethod "speak", 0 ; despacho dinamico por el type_name del objeto
Print                                           ; emite el String resultante
```

3.7. El *driver* y el contrato de evaluación (`hulkc`)

El binario `hulkc` orquesta el pipeline e implementa el contrato de `matcom/compilers`. La regla `make build` compila en modo *release* y copia el artefacto a `./hulk`. La invocación `./hulk archivo.hulk` valida las tres fases y, en caso de éxito, genera un ejecutable `./output`, `./output` se materializa como un *script* autocontenido que embebe la fuente validada y re-invoca el binario `hulk` en un modo interno `exec`; esta decisión reutiliza íntegramente el *backend* y satisface el contrato sin la complejidad de un generador a C o LLVM. Los errores se imprimen a *stderr* con el formato exacto `(line,col) TYPE: message`, con `TYPE ∈ {LEXICAL, SYNTACTIC, SEMANTIC}`, y el código de salida (1, 2 o 3) refleja el error más fundamental encontrado, regla que emerge naturalmente del orden de las fases.

3.8. Un ejemplo de extremo a extremo

Para ilustrar cómo cooperan las fases, considere el programa del Listado 1.5, que combina herencia, despacho virtual y `base`.

Cuadro 4. Resumen de las extensiones y sus referentes en lenguajes históricos.

Extensión	Estrategia	Referente
Genericidad	<i>type erasure</i> , invarianza	Java, Scala (<i>vs</i> Rust: monomorfización)
Interfaces	nominal + estructural	Java (nominal) + Go (estructural)
GC	<i>mark-and-sweep</i> , <i>handles</i>	Lisp, Lua (<i>vs</i> Python: <i>refcount</i>)

Listing 1.5. Programa HULK que ejercita herencia y despacho al padre.

```

type Animal(name: String) {
  name: String = name;
  speak(): String => self.name @ " hace un ruido";
}
type Dog(name: String) inherits Animal(name) {
  speak(): String => base() @ " ... guau!";
}
print((new Dog("Toby")).speak()); // Toby hace un ruido ... guau!

```

El flujo es el siguiente. (1) El **lexer** produce los tokens `type`, `Animal`, `(`, `...`, clasificando `inherits` y `base` y anotando cada uno con su *span*. (2) El **parser** construye el `Program` con dos `TypeDecl` y la expresión principal; la concatenación `@` se anida según su nivel de precedencia y `base()` se representa como una llamada especial al método homónimo del padre. (3) El **análisis semántico** registra `Animal` y `Dog` en `collect`, llena sus firmas en `sign`, comprueba en `check_overrides` que `Dog::speak` tiene la misma firma `(): String` que `Animal::speak`, y en `check_bodies` verifica que `self.name` es `String`, que `base()` resuelve a `Animal::speak` y que el argumento de `print` conforma a lo esperado. (4) El **descenso a IR** genera *bytecode* de pila: instrucciones para instanciar `Dog`, invocar `speak` con despacho dinámico y resolver `base` estáticamente al método del padre. (5) La **VM** ejecuta el *bytecode*: aloca el objeto `Dog` en el *heap* (devolviendo un `ObjectId`), despacha `speak` por el `type_name` concreto, ejecuta la concatenación y emite la salida `Toby hace un ruido ... guau!`. Si el programa tuviera un error —por ejemplo, si `Dog::speak` declarara un retorno `Number`— la fase 3 lo reportaría como (6,5) SEMANTIC: method speak overrides parent with a different signature y el proceso terminaría con código 3, sin alcanzar el *backend*.

4. Extensiones originales al lenguaje

Esta sección —núcleo del aporte de Lenguajes de Programación— define, analiza y demuestra las tres extensiones. Todas se implementaron como *commits* independientes y *backwards-compatible*: todo programa HULK estándar sigue funcionando sin cambios. La Tabla 4 sintetiza las decisiones de diseño y sus referentes históricos.

4.1. Genericidad: polimorfismo paramétrico por borrado de tipos

Definición y sintaxis. Un parámetro de tipo se declara entre corchetes tras el nombre del tipo o función y se usa como cualquier otro tipo en parámetros, atributos y retornos. La instanciación aporta los argumentos de tipo concretos:

Listing 1.6. Tipos y funciones genericidad en HULK.

```
type Box[T](item: T) {
  item: T = item;
  get(): T => self.item;
  set(v: T): T => self.item := v;
}
type Pair[A, B](a: A, b: B) { a: A = a; b: B = b; }
function id[T](x: T): T => x;

let n: Number = (new Box[Number](42)).get() in print(n); // 42
```

Reglas semánticas. El sistema introduce dos variantes de tipo: `Type::Generic(name, args)` para una instanciación como `List[Number]`, y `Type::Param(name)` para un parámetro `T` dentro del cuerpo de una declaración genérica. Las reglas son:

- **Invarianza:** `List[Animal]` *no* conforma a `List[Object]`. Se exige igualdad exacta de argumentos. Esta elección evita las sorpresas de varianza en un lenguaje didáctico y coincide con el comportamiento de Java para genericidad crudos.
- **Borrado en runtime:** `new List[Number]()` se ejecuta como `new List()`; la VM no ve los argumentos de tipo.
- **Chequeo de aridad:** un número incorrecto de argumentos de tipo produce un error `Arity` en compilación.
- **Sustitución:** en `(new Box[Number](42)).get()`, el retorno `T` se sustituye por `Number` mediante `substitute`.
- **Type::Param conforma solo a sí mismo y a Object.**

Implementación. El lexer aporta `[/]`; el AST añade `TypeRef::Generic(String, Vec<TypeRef>)` y un campo `generic_params` en `TypeDecl` y `FunctionDecl`; el parser introduce las reglas `GenericParams/GenericArgs`; y la semántica añade `resolve_type_ref_in_scope` y `substitute`. El IR y la VM *no* cambian estructuralmente, justamente porque el borrado de tipos hace que la representación en ejecución sea independiente de los argumentos genericidad.

Sustitución en acción. La regla de sustitución es la que da utilidad práctica a los genericidad. Al verificar `(new Box[Number](42)).get()`, el analizador instancia el cuerpo de `Box[T]` con el mapa $\{T \mapsto \text{Number}\}$: el atributo `item: T` pasa a `Number`, el constructor exige un argumento `Number` (y `42` conforma), y el retorno `get(): T` se reescribe a `get(): Number`. Así, la expresión completa recibe tipo `Number` y el enlace `let n: Number = ...` verifica sin coerciones. Si en cambio

se escribiera `new Box[Number]("x")`, la sustitución haría que `"x": String` no conforme al `Number` esperado por el constructor, produciendo un `Mismatch` con el *span* del argumento. La invarianza impide, además, pasar un `Box[Dog]` donde se espera un `Box[Animal]`, aun cuando `Dog ≤ Animal`.

Comparativa. Java y Kotlin también usan *erasure*, con notación `<T>`; Scala y Nim emplean corchetes `[T]` como nuestra extensión. Rust, en cambio, *monomorfiza*: genera código especializado por instanciación, más eficiente pero con mayor volumen de código [12,4]. La elección del *erasure* fue deliberada: encaja con una VM dinámicamente tipada sin requerir monomorfización, manteniendo el *backend* intacto.

4.2. Interfaces: contratos con conformidad nominal y estructural

Definición y sintaxis. Una interfaz declara un conjunto de firmas de métodos; puede heredar de otras (*extends*) y un tipo puede declarar que la implementa (*implements*), combinable con *inherits*:

Listing 1.7. Interfaces, herencia múltiple entre interfaces y despacho dinámico.

```
interface Ord { compare(): Number; }
interface Eq { equals(): Boolean; }
interface Comparable extends Ord, Eq { min(): Number; }

type Person(name: String) implements Greeter {
  name: String = name;
  greet(): String => "hi, " @ self.name;
}
let g: Greeter = new Person("kevin") in print(g.greet()); // hi, kevin
```

Reglas semánticas. Las interfaces incorporan una pasada `check_interfaces` (entre `sign` y `check_overrides`) y un mapa `InterfaceInfo` en el contexto de tipos. Las reglas clave son:

- **Implementación obligatoria:** `T implements I` exige que `T` (o un ancestro) provea todos los métodos de `I` —y los heredados vía *extends*— con la firma exacta.
- **No instanciable:** `new Greeter()` es error; solo los tipos concretos se instancian.
- **implements solo sobre interfaces:** `type X implements Y` con `Y` concreto produce `NotAnInterface`.
- **Herencia múltiple** entre interfaces y **implementación múltiple** (*implements* `A, B`); las clases heredan de una sola.
- **Borrado en runtime:** el IR/VM no ven interfaces; el despacho dinámico existente resuelve los métodos por el `type_name` del objeto concreto, sin *vtable* separada.

Conformidad dual. A diferencia de la descripción inicial puramente nominal, la implementación final de `conforms` (Listado 1.8) acepta *dos* vías cuando el tipo destino es una interfaz: la **nominal** (`implements` explícito, posiblemente transitivo vía herencia de tipos o `extends` de interfaces) y la **estructural** (*protocol-style*, A.10.2): el tipo provee todos los métodos requeridos con firma compatible aunque no declare `implements`. Esta dualidad sitúa a HULK entre el modelo nominal de Java/Kotlin/Swift —que evita acoplamientos accidentales y produce mensajes claros— y el estructural de Go/TypeScript —que favorece el desacoplamiento—, ofreciendo lo mejor de ambos de forma controlada.

Listing 1.8. Conformidad hacia una interfaz: primero la vía nominal, luego la estructural.

```
// dentro de conforms(a, b): cuando 'b' nombra una interfaz
if self.is_interface(&target_base) {
    if self.implements_interface(&cur_base, &target_base) {
        return true; // nominal: 'implements' (transitivo)
    }
    return self.structurally_implements(&cur_base, b); // estructural (A.10.2)
}
```

Implementación. El lexer aporta `interface/implements/extends`; el AST añade `InterfaceDecl` e `InterfaceMethodSig`, un campo `interfaces` en `Program` e `implements: Vec<TypeRef>` en `TypeDecl`; el parser, las reglas correspondientes; y la semántica, `lookup_interface_method`, `implements_interface`, `structurally_implements` e `interface_extends`. El IR y la VM permanecen intactos: las interfaces son borradas.

4.3. Recolector de basura *mark-and-sweep*

Motivación. La gestión previa, basada en `Rc<RefCell<Object>`, no podía liberar ciclos de objetos (el conteo de referencias nunca llega a cero en un ciclo). Para reclamarlos correctamente —y para ejercitar de primera mano un algoritmo clásico de gestión de memoria— se reemplazó por un recolector **mark-and-sweep** con un *heap* explícito.

Modelo de memoria. Los objetos viven en un `Heap` (un `Vec<Slot>`, con `Slot = Live(Object) | Free`) y se referencian por *handles* `ObjectId(u32)` que indexan el vector. Un `free_list` recicla los huecos liberados antes de crecer. El umbral de colección (`gc_threshold`, por defecto 1024) es configurable vía `HULK_GC_THRESHOLD`.

Listing 1.9. El *heap* gestionado por el GC.

```
pub struct Heap {
```

asignaciones 1..8	umbral alcanzado	tras collect()
+---+---+---+---+ B1 B2 ... B8 +---+---+---+---+ (todas Live)	+---+---+---+---+ B1 B2 ... B8 +---+---+---+---+ mark desde raíces: solo B8 (la actual) es alcanzable	+---+---+---+---+ Free Free ... B8 +---+---+---+---+ B1..B7 inalcanzables -> Free (al free_list) B8 (viva) permanece

Figura 4. Traza del *heap* en `gc.hulk` con umbral 8: al disparar la colección, solo la caja de la iteración en curso es alcanzable desde las raíces; las anteriores se reciclan al `free_list`.

```

slots: Vec<Slot>,          // Slot = Live(Object) | Free
free_list: Vec<u32>,       // huecos reutilizables (ids
                             reciclados)
allocations_since_gc: usize,
pub gc_threshold: usize,   // default 1024; HULK_GC_THRESHOLD
}
// Value::Object(ObjectId) reemplaza al antiguo Rc<RefCell<Object>>

```

Algoritmo.

1. **Mark:** recorrido en profundidad desde las raíces (la pila de operandos y los *scopes* de la VM). Todo `Value::Object(id)` se encola, se marca como vivo y se siguen recursivamente sus campos.
2. **Sweep:** los *slots* Live no marcados pasan a Free y su *id* se devuelve al `free_list` para reuso.
3. **Disparo:** tras cada asignación, si las alocaiones acumuladas alcanzan el umbral, se invoca `collect` con el conjunto de raíces.

Propiedades. El colector **reclama ciclos** (al partir de raíces vivas, un ciclo inalcanzable nunca se marca); **recicla slots** mediante el `free_list`; no compacta (el `Vec` crece monótonamente); es **backwards-compatible** (la salida observable es idéntica); y es **configurable** (`HULK_GC_THRESHOLD=1` fuerza GC por cada alocaión, útil en pruebas). Las cadenas se gestionan por el *ownership* normal de Rust y no participan en este *heap*.

Traza de ejecución. El caso `tests/extension/gc.hulk` ejecuta un bucle de cien iteraciones que crea en cada vuelta una caja efímera `Box(i)`, lee su valor y la descarta sin retener referencia. Con `HULK_GC_THRESHOLD=8`, el *heap* crece hasta el umbral y se libera repetidamente sin perder corrección (Figura 4). Como las cajas de iteraciones previas son inalcanzables desde las raíces —ya no hay variable que las apunte—, el *mark* no las visita y el *sweep* las recicla, manteniendo el *heap* acotado pese a las cien asignaciones. Una estrategia de conteo de referencias daría el mismo resultado en este caso acíclico; la diferencia se aprecia en `a.next := b; b.next := a`, donde solo el *tracing* reclama el ciclo.

Cuadro 5. Familias de recolección de basura comparadas en los ejes relevantes para un compilador educativo.

Eje	Conteo de ref.	Mark-and-sweep	Generacional
Reclama ciclos	no (sin extra)	sí	sí
Pausas	incrementales	<i>stop-the-world</i>	cortas (menores)
<i>Throughput</i>	medio	medio	alto
Complejidad de impl.	baja	baja-media	alta
Ejemplo	CPython	Lisp, Lua	JVM, Go

Comparativa. CPython usa principalmente conteo de referencias con un detector de ciclos adicional al estilo Bacon–Rajan [3]; nuestra extensión se simplifica usando solo *tracing*. Lua emplea un mark-and-sweep incremental [9]; nosotros usamos la versión *stop-the-world* por claridad pedagógica. Java y Go usan colectores generacionales y concurrentes [11], fuera del alcance de un compilador educativo. Rust prescinde de GC mediante *ownership* estático [12]; HULK lo necesita porque permite mutación compartida a través de objetos. El algoritmo elegido es, en esencia, el que McCarthy introdujo para Lisp en 1960 [16]. La Tabla 5 contrasta las tres familias según los ejes que motivaron nuestra elección. El *mark-and-sweep* ofrece el mejor compromiso para nuestros fines: reclama ciclos (lo que el conteo de referencias no logra sin un detector adicional) con una complejidad de implementación abordable (a diferencia de los colectores generacionales o concurrentes), a cambio de pausas *stop-the-world* asumibles en un intérprete didáctico.

4.4. Vectores (A.12): más allá del mínimo

Aunque no son una extensión original —pertenecen a *HULK — The Book*—, los vectores quedan fuera del mínimo exigido (A.1–A.8) y atraviesan todo el pipeline igual que las extensiones, por lo que se documentan aquí. Incluyen el literal `[e0, e1, ...]`, la indexación `v[i]`, el método `size()`, la comprensión por generador `[expr | x in iterable]` y la notación de tipo `T[]`. El tipo de un literal es `T[]` con `T` el LCA de sus elementos; un vector conforma al iterable tipado (`T[] <= T*`) pero es invariante respecto de otros vectores. En *runtime* el vector es un objeto del *heap* con una lista nativa y un cursor que implementa `next/current/size`, de modo que participa en el GC como cualquier otro objeto. Hacen pasar la categoría *generators* de la suite oficial de la cátedra.

5. Evaluación, pruebas y calidad

El proyecto mantiene una batería de pruebas a varios niveles: *unit tests* dentro de cada módulo para la lógica privada; *integration tests* por *crate* para la API pública; y pruebas *end-to-end* que ejecutan programas HULK completos (la suite de la cátedra en `tests/hulk_std/` y la de la extensión en `tests/extension/`, con

casos específicos `generics.hulk`, `interfaces.hulk` y `gc.hulk`). Cada variante de `SemError` cuenta con al menos una prueba negativa que la dispara, siguiendo el patrón `assert!(matches!(err, SemError::Variant ...))`. La integración continua (GitHub Actions) ejecuta *build*, pruebas, `clippy` y `rustfmt` en cada *push*, con `RUSTFLAGS="-D warnings"`, de modo que todo aviso del compilador se trata como error y solo se integran cambios con CI en verde. Esta disciplina hizo aflorar, por ejemplo, errores sutiles de ordenamiento en el despacho de métodos y en las raíces del GC, corregidos antes de la integración.

6. Trabajo relacionado

El diseño de *The Hitchhiker's Compiler* se inscribe en una larga tradición de compiladores e intérpretes didácticos. La estructura por fases y la separación *frontend/backend* siguen el modelo canónico de Aho et al. [1] y de Appel [2]. La elección de una máquina de pila con *bytecode*, en lugar de un árbol interpretado o un generador de código nativo, retoma el enfoque de *Crafting Interpreters* [17] y de las máquinas virtuales de Lua y la JVM, privilegiando la simplicidad de generación sobre el rendimiento absoluto.

En el plano del sistema de tipos, la conformidad por subtipado y el principio de sustitución remiten a Liskov y Wing [14], mientras que la taxonomía de polimorfismo que guía nuestra extensión de genericidad procede de Cardelli y Wegner [5] y su realización por *erasure* de la incorporación de genericidad a Java [4]. La decisión de ofrecer conformidad nominal y estructural de forma simultánea sitúa a HULK en un punto intermedio poco común entre el modelo estrictamente nominal de Java/Kotlin/Swift y el estructural de Go/TypeScript. Finalmente, el recolector de basura reconecta con la línea inaugurada por McCarthy [16] y sistematizada por Jones et al. [10,11], de la que tomamos la variante *mark-and-sweep* clásica. Lo distintivo de este trabajo no es la novedad de cada técnica por separado, sino su integración coherente y de bajo acoplamiento sobre una base modular en Rust que permitió añadir tres extensiones sin reescribir el núcleo.

7. Conclusiones

The Hitchhiker's Compiler implementa el pipeline completo de un compilador para HULK, desde el texto fuente hasta la ejecución, con un diseño modular que respeta SOLID y DRY, un análisis semántico robusto basado en acumulación de errores y el patrón *poison*, y una máquina virtual con recolección de basura. Sobre esa base, el equipo diseñó y verificó tres extensiones —genericidad, interfaces y un recolector *mark-and-sweep*— fundamentadas en un estudio comparativo con lenguajes históricos.

Ventajas. La arquitectura de dependencias unidireccionales y el AST como contrato único hicieron las extensiones notablemente baratas: genericidad e interfaces se añadieron *sin tocar* el IR ni la VM gracias al borrado de tipos, y el GC se aisló en un módulo `heap.rs`. El principio abierto/cerrado se cumplió en la práctica: casi todo el crecimiento fue mediante nuevas variantes de `enum` y nuevas pasadas, no modificando lógica existente. El manejo de errores por acumulación ofrece una experiencia de usuario superior a la de detenerse en el primer error.

Desventajas y críticas. El GC es *stop-the-world* y no compacta, por lo que el *heap* crece monótonamente y puede fragmentarse. El borrado de tipos, cómodo, sacrifica la información genérica en *runtime* (no hay *reified generics*). Persisten casos acotados de inferencia de tipos que difieren del comportamiento ideal del libro, y los mensajes de error sintáctico, aunque posicionados, son menos específicos que los semánticos. Tampoco se soportan comentarios de bloque.

Comparativa global. Frente a lenguajes consolidados, HULK extendido adopta decisiones didácticamente defendibles: el *erasure* de Java por simplicidad de *backend*; la conformidad dual (nominal de Java + estructural de Go) como punto medio entre seguridad y flexibilidad; y el *tracing* de Lisp/Lua frente al *refcounting* de Python, ganando la recuperación de ciclos a cambio de pausas. Ninguna de estas elecciones pretende competir en rendimiento con implementaciones de producción; todas privilegian la claridad conceptual.

Trabajo futuro. Las líneas naturales de mejora incluyen: (i) un *backend* de generación de código a C o LLVM, sustituyendo el intérprete; (ii) varianza acotada (*covariant/contravariant*) y cotas en los parámetros de tipo, superando la invarianza actual; (iii) un GC generacional o incremental para reducir las pausas; (iv) diagnósticos sintácticos con recuperación de errores y sugerencias; y (v) *reified generics* opcionales para habilitar reflexión de tipos en *runtime*. En conjunto, el proyecto demuestra que una base sólida y modular convierte la incorporación de características avanzadas de lenguajes en un ejercicio incremental y de bajo riesgo, validando la tesis planteada.

Disponibilidad. El código fuente completo, las pruebas y la documentación están disponibles en el repositorio del proyecto [20].

Referencias

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.
2. Andrew W. Appel. *Modern Compiler Implementation in ML*. Cambridge University Press, Cambridge, 1998.
3. David F. Bacon and V. T. Rajan. Concurrent cycle collection in reference counted systems. *Lecture Notes in Computer Science*, 2072:207–235, 2001.
4. Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler. Making the future safe for the past: Adding genericity to the Java programming language. In *Proceedings of OOPSLA '98*, pages 183–200. ACM, 1998.
5. Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.
6. George E. Collins. A method for overlapping and erasure of lists. *Communications of the ACM*, 3(12):655–657, 1960.
7. Frank DeRemer and Thomas Pennello. Efficient computation of LALR(1) look-ahead sets. *ACM Transactions on Programming Languages and Systems*, 4:615–649, 1982.
8. Universidad de La Habana Facultad de Matemática y Computación. HULK — the book: Havana university language for kompilers, 2024. Material docente del curso de Compilación.
9. Roberto Ierusalimsky. *Programming in Lua*. Lua.org, 4th edition, 2016.
10. Richard Jones, Antony Hosking, and Eliot Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, Boca Raton, FL, 2011.
11. Richard Jones and Rafael Lins. *Garbage Collection: Algorithms for Automatic Dynamic Memory Management*. John Wiley and Sons, Chichester, 1996.
12. Steve Klabnik and Carol Nichols. *The Rust Programming Language*. No Starch Press, San Francisco, CA, 2nd edition, 2023.
13. Donald E. Knuth. On the translation of languages from left to right. *Information and Control*, 8(6):607–639, 1965.
14. Barbara H. Liskov and Jeannette M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16:1811–1841, 1994.
15. Niko Matsakis and Contributors. LALRPOP: LR(1) parser generator for Rust, 2024. <https://github.com/lalrpop/lalrpop>.
16. John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part I. *Communications of the ACM*, 3(4):184–195, 1960.
17. Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021.
18. Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.
19. Maciej Płesak and Contributors. Logos: Create ridiculously fast lexers, 2024. <https://github.com/maciejhirszyk/logos>.
20. Kevin Alejandro Torres Perera, Lianny de la Caridad Reveal Valdivieso, and Jocdan Lismar López Mantecón. The hitchhiker's compiler, 2026. <https://github.com/MatCom-Developing-Team-10/The-Hitchhikers-Compiler>.